

Modul- Fortgeschrittene Programmierkonzepte

Bachelor Informatik

14 - Zusammenfassung

Prof. Dr. Marcel Tilly

Fakultät für Informatik, Cloud Computing

Today

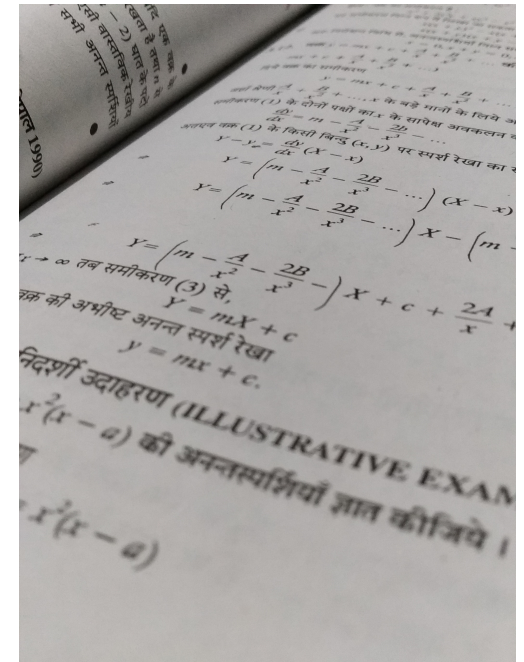
Keine Besprechung der Musterlösung in der Übung

Übung unter: <https://inf-git.fh-rosenheim.de/inf-fpk/13-map-reduce-collect>

Agenda for today!



What is hot and what is not!



Aus dem Modulhandbuch

Die Studierenden ...

- ... **vertiefen** ihre Kenntnisse in der objektorientierten Programmierung am Beispiel einer geeigneten Programmiersprache (hier: Java!)
- ... können die Möglichkeiten und Gefahren der objektorientierten Programmierung **beurteilen**.
- ... **sind befähigt**, alle wichtigen Programmierkonzepte für das Programmieren im Großen im Sinne der Komponentenorientierung anzuwenden.
- ... **erarbeiten sich die Grundlagen** der funktionalen Programmierung und deren Anwendungsgebiete.



OOP Paradigm

OOP Characteristics

OOPs Concepts:

- Polymorphism
- Inheritance
- Encapsulation/ Information Hiding
- Abstraction
- Class
- Object
- Method
- Message Passing

- *Information hiding* (or *encapsulation*) is a fundamental concept in object-oriented programming.
- In Java, this is realized using
 - *interfaces* (keyword `interface`): describe the externals of modules
 - *classes* (keyword `class`): encapsulate information (variables) and business logic (methods).
- Also, grouping information and algorithms to coherent modules

Implements vs. Extends

```
class Rectangle extends Shape implements Drawable {
    private int width, height;
    Rectangle(int x, int y, int w, int h) {
        super(x, y);
        width = w;
        height = h;
    }
    public void draw(Canvas c) { /* do some magic */ }
}
```

- The **Rectangle** literally **extends** a general **Shape**: aside from **x** and **y** coordinates, it is defined by **width** and **height**.
- The **Rectangle** also **implements** **Drawable**: given some **Canvas**, it can draw itself.

When to use what?

Following the semantics of the keywords, you should

- *extend* a class, when you aim to make something more specific; a `Rectangle` will always be a `Shape`.
- *implement* an interface, when you aim to extend a class by certain (potentially orthogonal) functionality; not every `Shape` might be drawable, and there might be other classes which happen to be drawable.

Multiple Inheritance

One solution is to define Van and Pickup as interface:

```
interface Van {  
    void board(Person p);  
    void unboard(Person p);  
}  
  
interface Pickup {  
    void load(Cargo c);  
    void unload(Cargo c);  
}  
  
class VwTransporterPickup implements Van, Pickup {  
    List passengers, cargos;  
    void board(Person p) {  
        passengers.add(p);  
    }    // ...  
}
```

But this requires us to implement all the methods explicitly!

Multiple Inheritance

- default methods in interfaces is possible now!
- private methods in interfaces is possible now!

```
interface Van {  
    List getPersons();  
    default void board(Person p) {  
        getPersons().add(p);  
    }  
    // ...  
}  
  
class VwTransporterPickup implements Van, ... {  
    private List persons;  
    public List getPersons() {  
        return persons;  
    }  
    // ...  
}
```

The Diamond Problem

```
class Bottom implements Left, Right {  
    public void method() {  
        System.out.println("Bottom.method()");  
        Left.super.method();  
        Right.super.method();  
    }  
  
    public static void main(String... args) {  
        Bottom b = new Bottom();  
        b.method();  
    }  
}
```

- The interfaces `Left` and `Right` add default implementations for `method()`.
- Use `super.<method>` to access the implementation of a **base class**, but use `<Interface>.super.<method>` to access default methods.

Note that commenting out `Bottom.method()` will lead to a compiler error!

- **Mixin programming** is a style of software development where units of functionality are created in a class and then mixed in with other classes (see **Aspect Oriented Programming**).
- A mixin class is a parent class that is inherited from - but not as a means of specialization. Typically, the mixin will export services to a child class, but no semantics will be implied about the child "being a kind of" the parent.
- Mixin can provide other services based on the services the original class provides.

taken from <http://wiki.c2.com/?Mixin>

Conceptually, the difference between inheritance and mixins is that the latter are meaningless on its own and can be attached to other unrelated classes.

On the other hand, inheritance is used if there is a strong relation between the classes.

The main issue with the above realization of mixins is the lack of *state*: since the interfaces cannot have attributes, the only way to read/write data would be through (public) setters/getters.

Example: Let's say, you want to gradually escalate your shouting.

```
Message m = new Message("Hello, World");  
m.escalate(); // "HELLO, WORLD"  
m.escalate(); // "HELLO, WORLD!"  
m.escalate(); // "HELLO, WORLD!!"  
m.escalate(); // "HELLO, WORLD!!!"
```

Generics

“Java Generics are a language feature that allows for definition and use of generic types and methods. Generic types are instantiated to form parameterized types by providing actual type arguments that replace the formal type parameters.”

Instead of using `Object` along with type casts, use type parameters (type variables) as placeholder for actual types, i.e. instead of `Object`, use `T`. The type parameters need to be declared in the signature of the class, in `<...>` and between the name and the opening curly parenthesis:

```
interface Map<K, V> {  
    void put(K key, V value);  
    V get(K key);  
}
```

While you could use any identifier for the type parameters, it is customary to use single letters. Use `T` for a single type, `K` and `V` for key and value, `R` and `S` for unrelated types (see towards the end of this class). If you need to use multiple type parameters, separate them with comma.

For methods, the type parameters are specified prior to the return type, and type parameters can be used both for arguments and return types.

```
class Example {  
    static <T> T[] reverse(T[] in) {  
        T[] clone = in.clone();  
        for (int i = 0; i < in.length/2; i++)  
            swap(clone, i, in.length - 1 - i);  
        return clone;  
    }  
    private static <T> void swap(T[] arr, int i, int j) {  
        T h = arr[i];  
        arr[i] = arr[j];  
        arr[j] = h;  
    }  
    public static void main(String... args) {  
        Integer[] arr = {1, 2, 3, 4, 5};  
        Integer[] rev = reverse(arr); // type safety at compile time!  
        // will produce error at compile time! (Integer[] and String[] incompatible)  
        Integer[] oha = (Integer[]) reverse(  
            new String[] {"Hans", "Dampf"});  
    }  
}
```

To enforce that a certain class is either a subclass or implements a certain interface, use the following syntax with `extends`:

```
class SortedMapImpl<K extends Comparable<K>, V> implements Map<K, V> {  
    // ...  
    public V get(K key) {  
        Entry it = root;  
        while (it != null) {  
            // no cast necessary!  
            int c = key.compareTo(it.key);  
            if (c == 0) return it.value;  
            else if (c < 0) it = it.left;  
            else it = it.right;  
        }  
    }  
}
```

- To enforce more than one class or interface, use the `&` symbol (since the `,` is already reserved for multiple type arguments), for example: `<T extends Comparable<T> & Serializable>`.

Wildcards and Bounds

- Which is not the supertype for all kinds of `Collections`?
- What is the supertype of all `Collections`?

It is a `Collection` with *unknown* type, which is denoted using the wildcard `?`:

```
void print(Collection<?> c) {  
    for (Object o : c) {  
        System.out.println(o);  
    }  
}
```

Wildcard ? extends

- Since we don't know what the element type of `c` stands for, we cannot add objects to it.
- The `add()` method takes arguments of type the collection is bound to.
- Any parameter we pass to `add` would have to be a subtype of this now unknown (?) type.
- Since we don't know what type that is, we cannot pass anything in.
- An exception is `null`, which is a member of every type.

Similar to type variables, wildcards can be bound.

```
class Klass {  
    void method() { /* ... */ }  
}
```

```
void apply(Collection<? extends KlassA> c) {  
    for (Klass k : c) {  
        k.method();  
    }  
}
```

An *upper* bound, defining that the class is unknown, but *at least* satisfies a certain class or interface.

For example, `List<Integer>` fits as a `List<? extends Number>`.

What is the difference between a wildcard bound and a type parameter bound?

1. A wildcard can have only one bound, while a type parameter can have several bounds (using the & notation).
2. A wildcard can have a *lower* or an upper bound, while there is no such thing as a lower bound for a type parameter.

So what are *lower* bounds?

A lower bounded wildcard restricts the unknown type to be a specific type or a supertype of that type.

In the previous examples with *upper bounds*, we were able to *read* (`get()`) from a collection, but not *write* (`add()`) to a collection. If you want to be able to *write* to a collection, use a *lower* bound:

```
void augment(List<? super Klass> list) {  
    for (int i = 1; i <= 10; i++) {  
        list.add(new Klass()); // this works  
    }  
    // compile time error: can't resolve type  
    Klass k = list.iterator().next();  
    // runtime hazard: ClassCastException  
    Klass k = (Klass) list.iterator().next();  
}
```

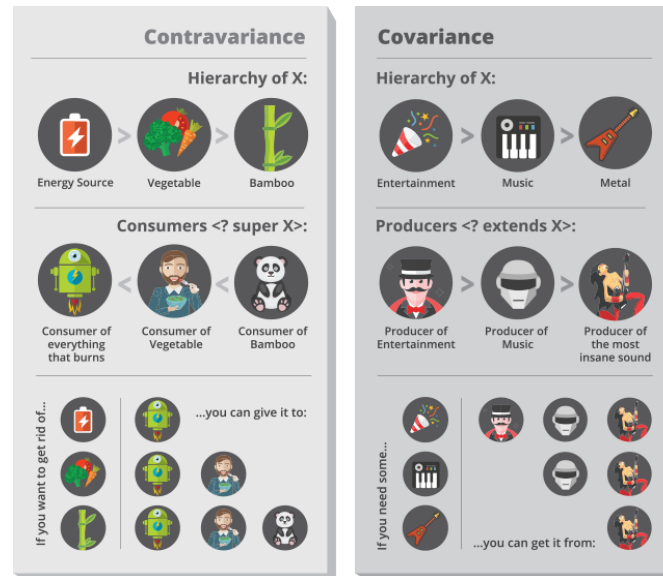
Definition: Liskov Substitution Principle:

if S is a subtype of T , then objects of type T may be replaced with objects of type S .

Within the type system of a programming language, a typing rule

- **Covariant** if it preserves the ordering of types ($\leq$), which orders types from more specific to more generic
- **Contravariant** if it reverses this ordering;
- **Invariant or nonvariant** if neither of these applies.

Languages supporting generics (such as Java or Scala, and to some extent C++), feature *covariance* and *contravariance*, which are best described in the following diagram by [Oleg Shelajev at RebelLabs](#) (based on a diagram by [Andrey Tyukin](#) available under the CC-BY-SA).



Case 1: You want to go through the collection and do things with each item.

- The list is a **producer**, so you should use a `Collection<? extends Thing>`.
- The reasoning is that a `Collection<? extends Thing>` could hold any subtype of `Thing`, and thus each element will behave as a `Thing` when you perform your operation.
- You actually cannot add anything to a `Collection<? extends Thing>`, because you cannot know at runtime which specific subtype of `Thing` the collection holds.

Case 2: You want to add things to the collection.

- The list is a **consumer**, so you should use a `Collection<? super Thing>`.
- The reasoning here is that unlike `Collection<? extends Thing>`, `Collection<? super Thing>` can always hold a `Thing` no matter what the actual parameterized type is.
- Here you don't care what is already in the list as long as it will allow a `Thing` to be added; this is what `? super Thing` guarantees.

The combination of the two principles (contravariance and covariance) is known as *PECS* -- *producer extends, consumer super*. The mnemonic is seen from the collection's point of view.

- If you are retrieving items from a generic collection, it is a producer and you should use *extends*.
- If you are adding items, it is a consumer and you should use *super*.
- If you do both with the same collection, you shouldn't use either *extends* or *super* (but a type variable, with bounds if needed).

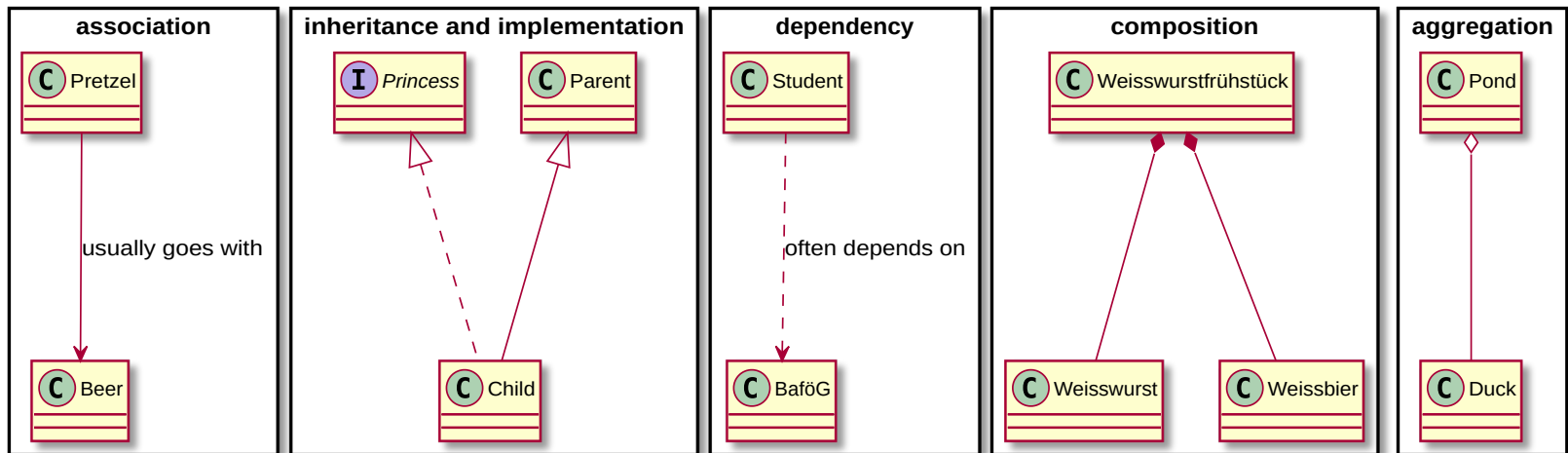
The classic example for PECS is a function that reads from one collection and stores them in another, e.g. copy:

```
static <T> void copy(Collection<? extends T> source,  
    Collection<? super T> dest) {  
    for(T n : source) {  
        dest.add(n);  
    }  
}
```



Design Pattern

Class Diagrams



Association: References a ...

Inheritance: *Is-A* relation

Implements: behavioral relation

Composition: real-world whole-part relation

Aggregation: "catalog" containment, can exist independently

Design Pattern



Structural Patterns

- Adapter pattern
- Bridge pattern
- Filter pattern
- Composite pattern
- Facade pattern
- Decorator pattern
- Proxy pattern
- Flyweight pattern

Creational Patterns

- Builder pattern
- Singleton pattern
- Prototype pattern
- Factory method
- Abstract factory

Behavioural Patterns

- Interpreter pattern
- Template pattern
- Visitor pattern
- Strategy pattern
- State pattern
- Observer pattern
- Memento pattern
- Iterator pattern
- Command pattern
- Mediator pattern
- Chain of responsibility



Parallel Programming

- To make the bean counters work in parallel, use the [Thread class](#).
- It takes an instance of `Runnable` of which it will execute the `.run()` method *in a separate thread*, once the thread's `.start()` method is called.
- First, modify the `BeanCounter` to implement the `Runnable` interface

```
public class BeanCounter implements Runnable {  
    // ...  
}
```

The Consumer-Producer Problem

The buffer provides the basic operations `put(T t)` and `T get()`, which should each block if full (put) or empty (get).

```
class Buffer<T> {  
    List<T> buffer = new LinkedList<>();  
    final int max = 10;  
  
    synchronized void put(T obj) throws InterruptedException {  
        // wait until buffer not full  
        while (buffer.size() == 10)  
            wait();  
  
        buffer.add(obj);  
  
        // wake up other threads waiting for buffer to change  
        notifyAll();  
    }  
}
```

The Consumer-Producer Problem

... continued

```
synchronized T get() throws InterruptedException {  
    // wait until there's something in the buffer  
    while (buffer.size() == 0)  
        wait();  
  
    T obj = buffer.remove(0);  
  
    // wake up other threads waiting for buffer to change  
    notifyAll();  
    return obj;  
}  
}
```

Working Asynchronously

A possible solution is to store both arguments and results inside the thread or `Runnable`, and relay possible exceptions to whoever tries to retrieve the result.

```
public class JobWithResult implements Runnable {
    private int a, b, result;
    private Exception e;

    JobWithResult(int a, int b) {
        this.a = a;
        this.b = b;
    }

    @Override
    public void run() {
        try {
            this.result = a / b;
        } catch (Exception e) {
            this.e = e;
        }
    }

    int getResult() throws Exception {
        if (e != null)
            throw e;
        return result;
    }
}
```

Callables, Futures and Executors since Java 5

Since Java 5, these three parts are realized as [Callable](#) (1), [Executor](#) (2) and [Future](#) (3). The "real" interfaces are slightly different to allow a more faceted use.

```
interface Callable<V> {  
    V call();  
}
```

```
interface Future<V> {  
    boolean cancel(boolean mayInterruptIfRunning);  
    V get();  
    V get(long timeout, TimeUnit unit);  
    boolean isCancelled();  
    boolean isDone();  
}
```

```
interface ExecutorService extends Executor {  
    void execute(Runnable command); // for convenience  
    <T> Future<T> submit(Callable<T> task);  
    Future<?> submit(Runnable task);  
    <T> Future<T> submit(Runnable task, T result);  
    // ...  
}
```



Functional Programming

Why Functional Programming?

So what's the big deal with functional programming?

1. Since objects are immutable, parallization is (almost) trivial (you may have heard of [map-reduce](#)).
2. *Separation of Concerns (SoC)*: FP helps you to separate the *data traversal* (how you *iterate* the data) from the *business logic* (what you *do* with the data).

Example

Say you want to

- retrieve all students from a database,
- filter out those who took *Programmieren 3*,
- load their transcript of records from another database
- print all class names

Iterative Solution

```
for (Student s : getStudents()) {  
    if (s.getClasses().contains("Programmieren 3")) {  
        ToR tor = db.getToR(s.getMatrikel());  
        for (Record r : tor) {  
            System.out.println(r.getName());  
        }  
    }  
}
```

A Simple Immutable List

`head` stores the data, `tail` links to the next element.
The end of the list is explicitly modeled.

```
class List<T> {  
    final T head;  
    final List<T> tail;  
  
    private List(T el, List<T> tail) {  
        this.head = el;  
        this.tail = tail;  
    }  
  
    boolean isEmpty() {  
        return head == null;  
    }  
  
    // ...  
}
```

Reduction

Working on lists (and later streams), we defined three main methods:

- `filter` utilizing a `Predicate<T>` to retain only certain elements,
- `map` utilizing a `Function<T, R>` to transform a list of elements of type `T` to a list of type `R`
- `forEach` utilizing a `Consumer<T>` that accepts (in order of the list).

Working on lists, we defined those *recursively*:

```
static <T> List<T> filter(List<T> xs, Predicate<T> p) {  
    if (xs.isEmpty()) return xs;  
    else if (p.test(xs.head)) return list(xs.head, filter(xs.tail, p));  
    else return filter(xs.tail, p);  
}
```

FP Recap

Can we remember how to formulate `map` and `forEach`:

Can we remember how to formulate **map** and **forEach**:

```
static <A, B> List<B> map(List<A> xs, Function<A, B> f) {  
    if (xs.isEmpty()) return empty();  
    else return list(f.apply(xs.head), map(xs.tail, f));  
}
```

Can we remember how to formulate **map** and **forEach**:

```
static <A, B> List<B> map(List<A> xs, Function<A, B> f) {  
    if (xs.isEmpty()) return empty();  
    else return list(f.apply(xs.head), map(xs.tail, f));  
}
```

```
static <A> void forEach(List<A> xs, Consumer<A> c) {  
    if (xs.isEmpty()) return;  
    else {  
        c.accept(xs.head);  
        forEach(xs.tail, c);  
        // return (added for clarity)  
    }  
}
```

Tail recursion

Alternative:

```
static int sum(List<Integer> xs, int z) {  
    if (xs.isEmpty()) return z;  
    else return sum(xs.tail, z + xs.head);  
}
```

which evaluates to

```
sum(list(1, 3, 3, 7), 0)  
-> sum(list(3, 3, 7), 0 + 1)  
-> sum(list(3, 7), 1 + 3)  
-> sum(list(7), 4 + 3)  
-> sum(empty(), 7 + 7)  
-> 14
```

Depending on the language, they can be realized as a for-loop reusing the stack variables.

Reduce

Let's isolate the operation, using the interface
`java.util.function.BinaryOperator<T>`:

```
interface BinaryOperator<T> {  
    T apply(T t1, T t2);  
}
```

```
static <T> T reduce(List<T>, T z, BinaryOperator<T> op) {  
    if (xs.isEmpty()) return z;  
    else return reduce(xs.tail, op.apply(z, xs.head), op);  
}
```



Streams

Generation

- `Stream.of(...)` with array or varargs
- `Collection.stream()`, if supported
- `Stream.generate(...)` using a generator
- Popular APIs, e.g. `Pattern.compile("\\W").splitAsStream("hello world");`



Intermediate Operations

You already know most of the intermediate operations:

- `filter(Predicate<T> p)` removes/skips unwanted elements in the stream
- `map(Function<T, R> f)` transforms a `Stream<T>` into a `Stream<R>` using the provided `Function`
- `sorted(Comparator<T> comp)` returns a sorted stream
- `concat(Stream<T> s)` appends another stream
- `distinct()` removes duplicates
- `skip(int n)` and `limit(int n)` skip elements and truncate the stream

Collectors 1/2

Another powerful tool provided by the Java Streams API is `collect` which is a special form of stream reduction.

The idea is to iterate over the stream and pass each element to a *combiner* that builds up a data structure. A classic example is to turn a `Stream` into a `List`:

```
List<Integer> list1 = new LinkedList<>();  
Stream.of(1, 3, 3, 7).forEach(i -> list.add(i));  
  
// or shorter, using collect  
List<Integer> list2 = Stream.of(1, 3, 3, 7).collect(Collectors.toList());
```

Collectors 2/2

Java provides a [lengthy list of collectors](#) for your convenience. Here are a few examples from the docs, most notably `groupingBy` und `partitioningBy`.

```
// Accumulate names into a TreeSet
Set<String> set = people.stream()
    .map(Person::getName)
    .collect(Collectors.toCollection(TreeSet::new));

// Convert elements to strings and concatenate them, separated by commas
String joined = things.stream()
    .map(Object::toString)
    .collect(Collectors.joining(", "));

// Compute sum of salaries of employee
int total = employees.stream()
    .collect(Collectors.summingInt(Employee::getSalary));

// Group employees by department
Map<Department, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Compute sum of salaries by department
Map<Department, Integer> totalByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
        Collectors.summingInt(Employee::getSalary)));

// Partition students into passing and failing
Map<Boolean, List<Student>> passingFailing = students.stream()
    .collect(Collectors.partitioningBy(s -> s.getGrade() <= 400));
```



Final words

What is important?

In case, you have only one day left ...

- Design Pattern
- Generics
- Recursion
- Functional Programming
- Parallel Programming/ Futures
- Classes/ Interfaces

Final Thought!

